WEEK 7 – VAPT WEB APPLICATION TESTING WITH BURP SUITE

This report describes the web application security testing activities performed in a local DVWA lab. The work included request interception, application mapping, basic vulnerability testing, and directory enumeration.

Introduction

Vulnerability Assessment and Penetration Testing (VAPT) helps identify security weaknesses in an application. In this practical, Burp Suite was used to inspect web requests and test selected vulnerabilities in Damn Vulnerable Web Application (DVWA).

Objectives

- Configure Burp Suite and intercept HTTP requests.
- Explore the application and identify request parameters.
- Test SQL Injection and Cross-Site Scripting (XSS) in the authorized lab.
- Use Dirsearch to identify accessible paths and files.
- Document evidence, possible impact, and recommended fixes.

Scope and Authorization

Target: DVWA running locally at http://127.0.0.1/DVWA/. Testing was restricted to the local practice application. The results in this report apply to this lab configuration and should not be treated as findings about an unrelated production website.

Note: DVWA is intentionally vulnerable. Low security mode was used for the demonstrated SQL Injection and XSS exercises.

Tools used

Burp Suite was used for HTTP request inspection and Repeater testing. Dirsearch was used for path discovery.

DVWA provided a controlled environment for practicing common web vulnerabilities.

Testing Environment and Tools

Component	Details
Operating system	Kali Linux
Target application	Damn Vulnerable Web Application (DVWA)
Target URL	http://127.0.0.1/DVWA/
Proxy and testing tool	Burp Suite Community Edition
Enumeration tool	Dirsearch
Browser	Burp Suite's built-in browse

Methodology

- Configure the proxy and intercept requests in Burp Suite.
- Browse DVWA and review its pages using the Target site map.
- Inspect request parameters and test inputs in the authorized lab.
- Run directory enumeration against the local DVWA URL.
- Record observed results and prepare remediation advice.

Task 22 – Burp Suite Setup and Request Interception

Burp Suite was configured to inspect traffic between the browser and the local DVWA application. The browser was used to open the DVWA login page, and Burp Suite intercepted the HTTP requests.

Steps performed

- Opened DVWA through the Burp Suite browser.
- Observed the HTTP request for the login page.
- Submitted the login form and inspected the login request.
- Used the Proxy interception feature to view request details.

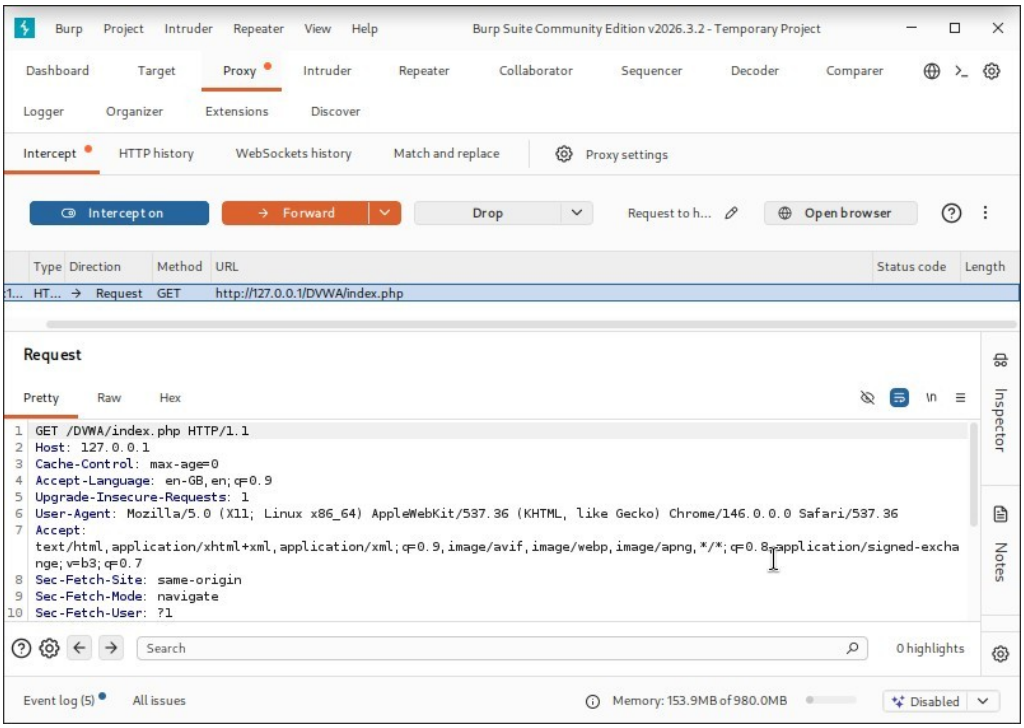


Figure 1. HTTP request intercepted in Burp Suite

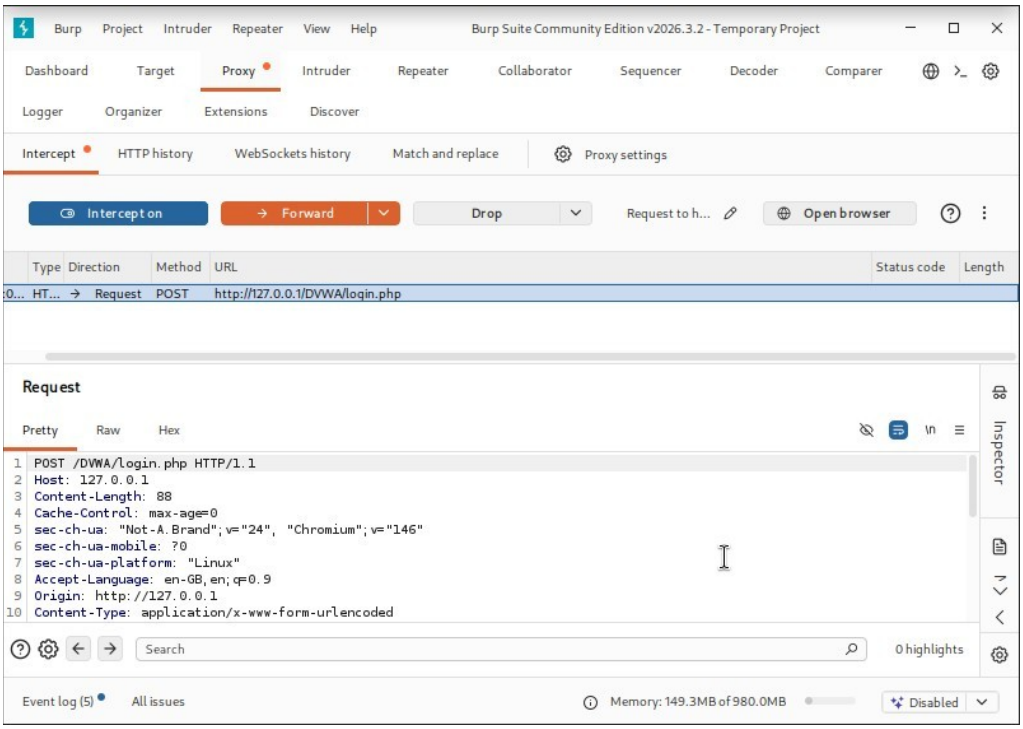


Figure 2. DVWA login request captured in Burp Suite.

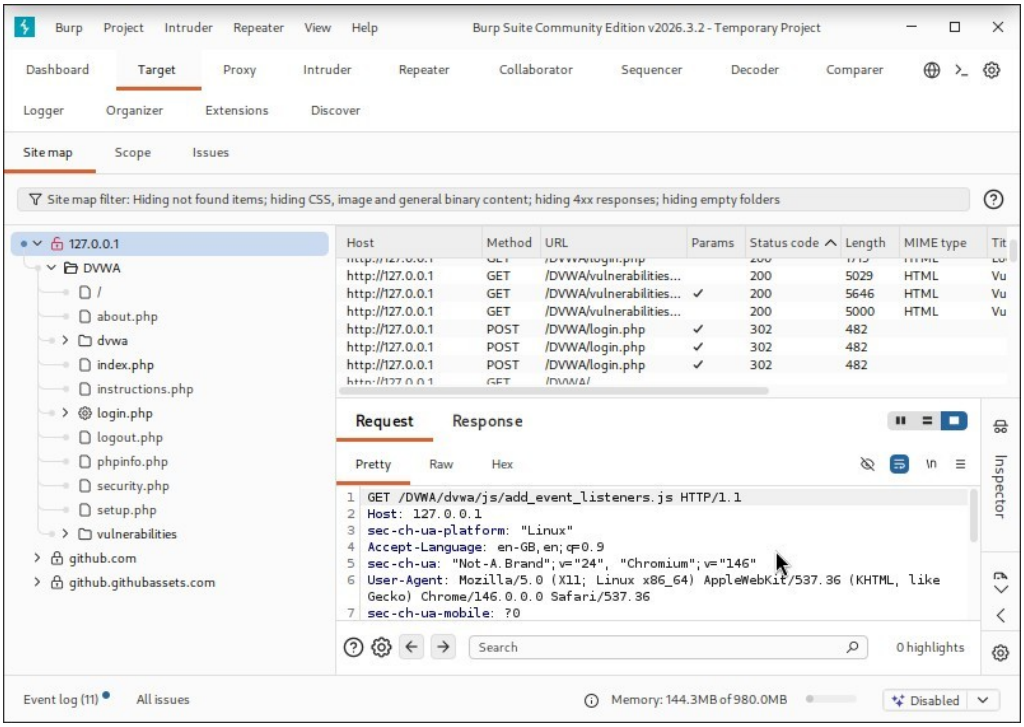
Result: The requests were visible in Burp Suite, allowing the request method, path, headers, and submitted login parameters to be inspected.

Task 23 – Recon + Mapping Web Application Use Burp Proxy + Target Tab

The DVWA site map in Burp Suite was used to explore pages and understand the application's request structure. The SQL Injection page was opened and a normal input was submitted to inspect its parameters.

Parameters observed

- **id:** The value submitted to the SQL Injection page.
- **Submit:** The form submission parameter.
- **user_token:** A token included in the request.



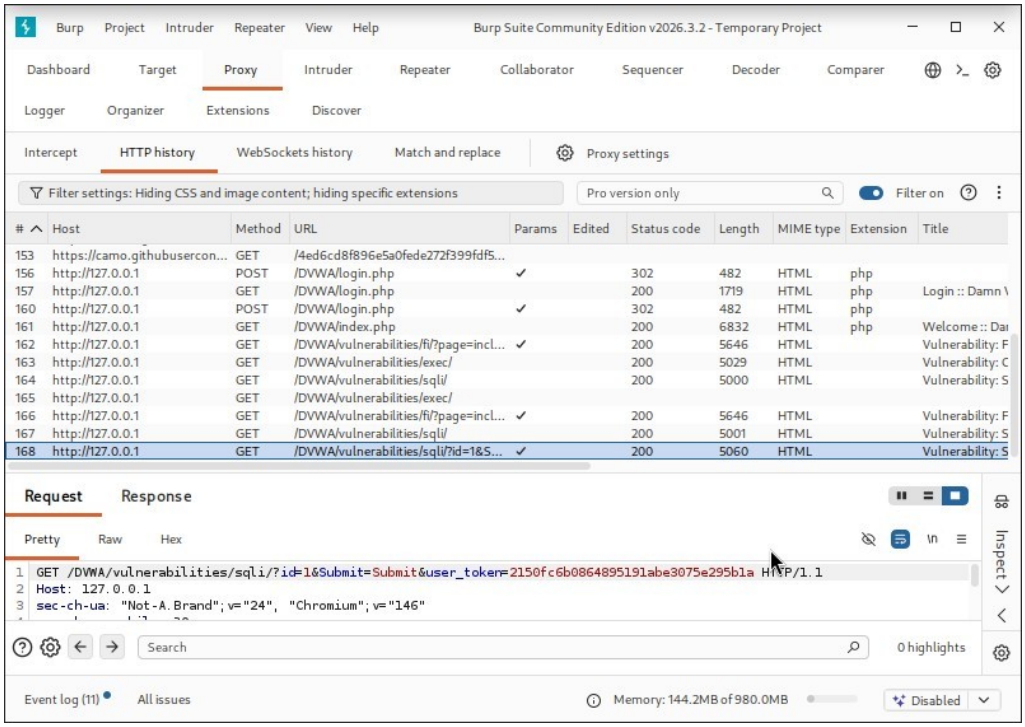


Figure 3. DVWA pages visible in the Burp Suite site map.

Figure 4. SQL Injection request parameters

Result: Application pages and request parameters were identified. This mapping helped select the input parameter for the next testing activity.

Task 24 – Vulnerability Testing (Manual + Burp Tools)

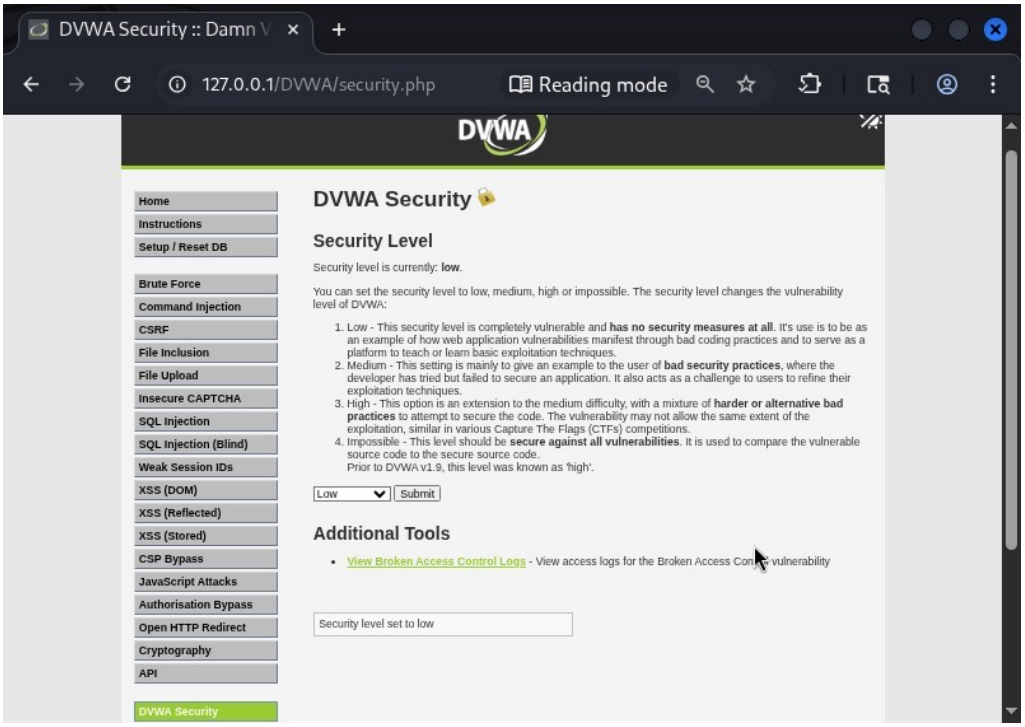
SQL Injection Testing

SQL Injection occurs when an application includes untrusted input in a database query without proper handling.

The DVWA SQL Injection page was tested at Low security level in the local lab.

Testing Steps

- Opened the SQL Injection module.
- Sent a normal request and reviewed the response in Burp Repeater.
- Modified the id parameter using the test payload: 1' OR '1'='1.
- Compared the response with the normal request.



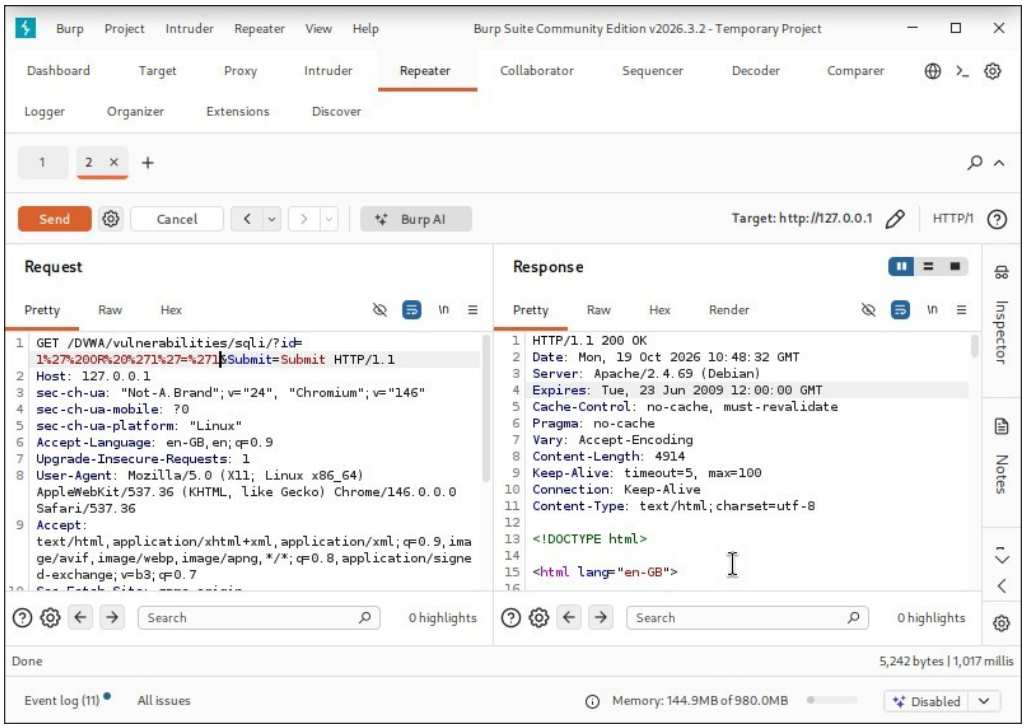


Figure 5. DVWA security level used for the exercise.

Observed result

The modified request returned multiple database records, including records for different users. This confirms that the input affected the query in the tested Low-security DVWA configuration.

Figure 6. SQL Injection test evidence.

Potential impact

In a real application, SQL Injection may allow unauthorized access to database information or changes to data, depending on the query and database permissions.

Recommended remediation

- Use parameterized queries or prepared statements.
- Validate input on the server side.
- Use a database account with minimum required permissions.
- Avoid displaying detailed database errors to users.

Reflected Cross-Site Scripting (XSS)

Reflected XSS occurs when an application returns untrusted input in a response without safely encoding it. A JavaScript test payload was submitted through the DVWA Reflected XSS page.

Testing Steps

- Opened the Reflected XSS module.
- Entered <script>alert(1)</script> in the input field.
- Submitted the form and observed the browser response.

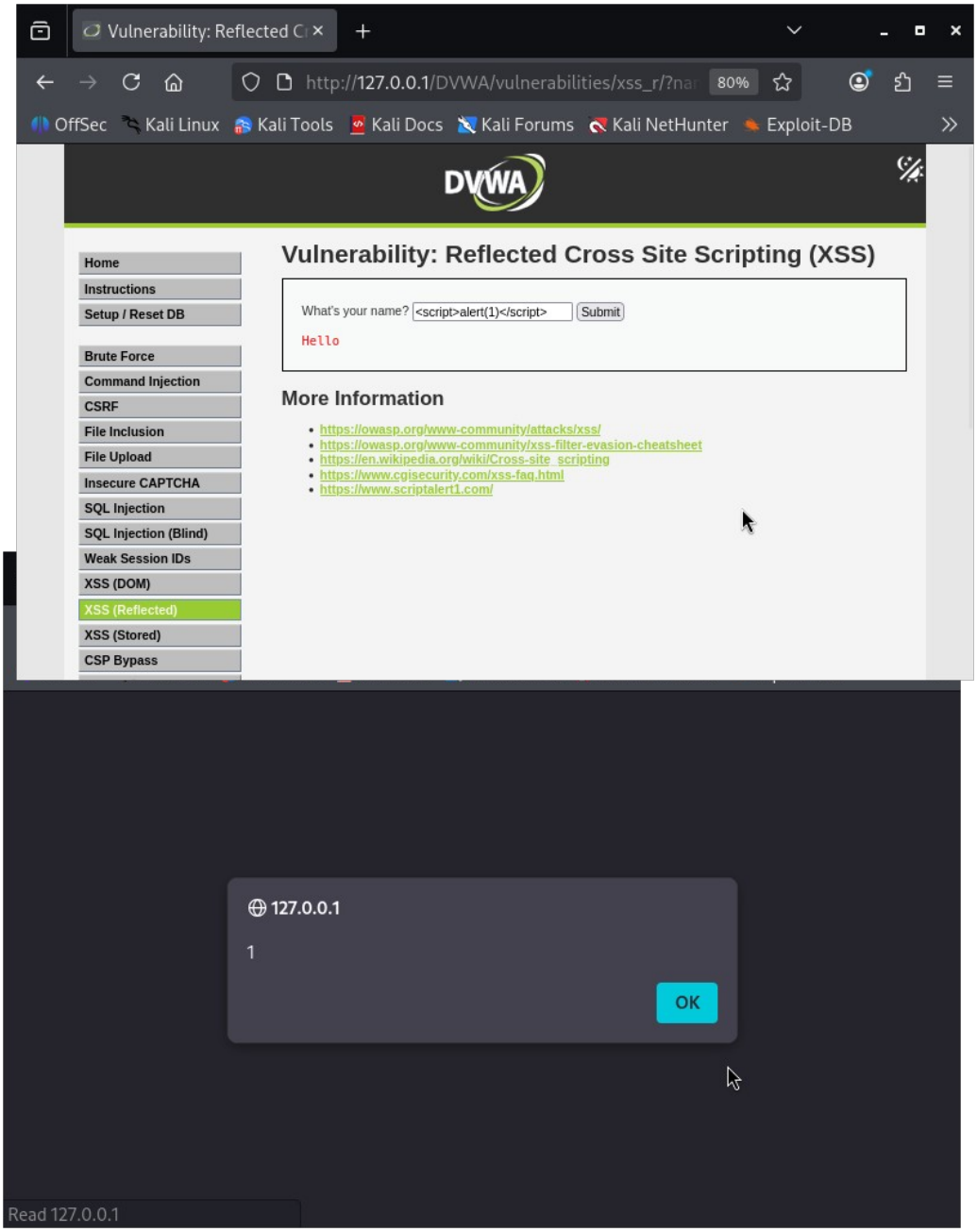


Figure 7.

JavaScript alert produced during the Reflected XSS test.

Figure 8. Reflected XSS page after submitting the test input.

Observed result

The browser displayed an alert containing the value 1. This confirms JavaScript execution from the submitted input in the tested DVWA configuration.

Potential Impact

If present in a real application, XSS could allow scripts to run in another user's browser in the application's origin, potentially exposing sensitive page data or performing actions as that user.

Recommended remediation

- Apply context-aware output encoding.
- Validate input where appropriate.
- Use safe templating methods and avoid inserting untrusted input as HTML.
- Use Content Security Policy as an additional defensive layer.

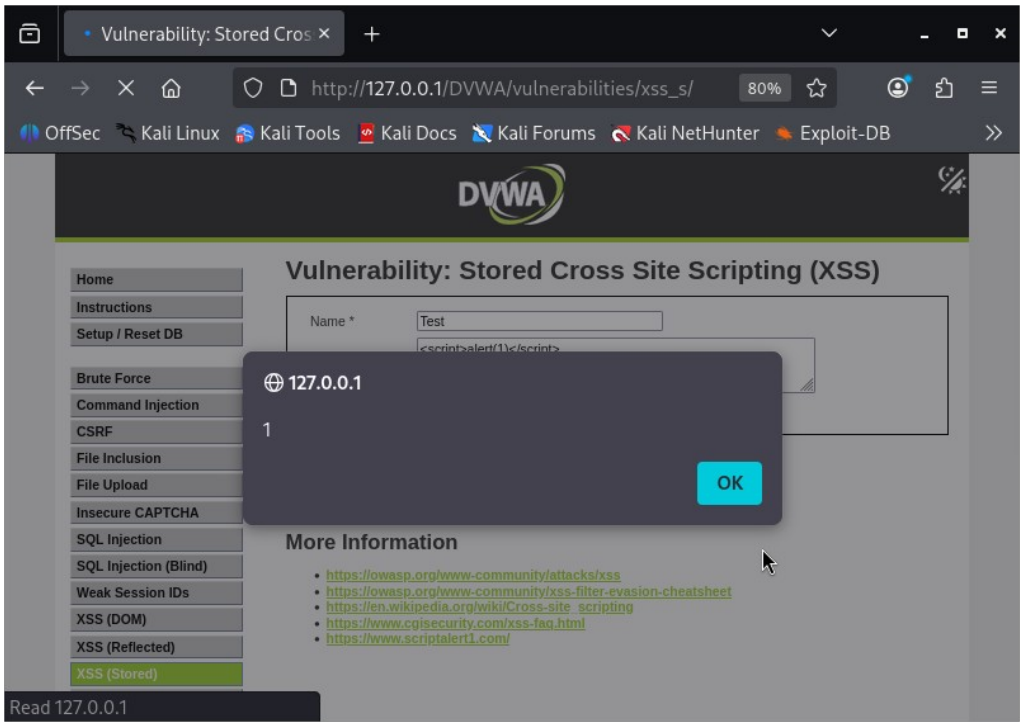
Stored Cross-Site Scripting (XSS)

Stored XSS occurs when an application saves untrusted input and later displays it in a way that executes as script. The DVWA guestbook form was used for this test.

Testing steps

- Opened the Stored XSS module.
- Entered a test name and submitted the JavaScript payload as the message.
- Observed the alert when the submitted guestbook entry was processed.

.



Alert the XSS

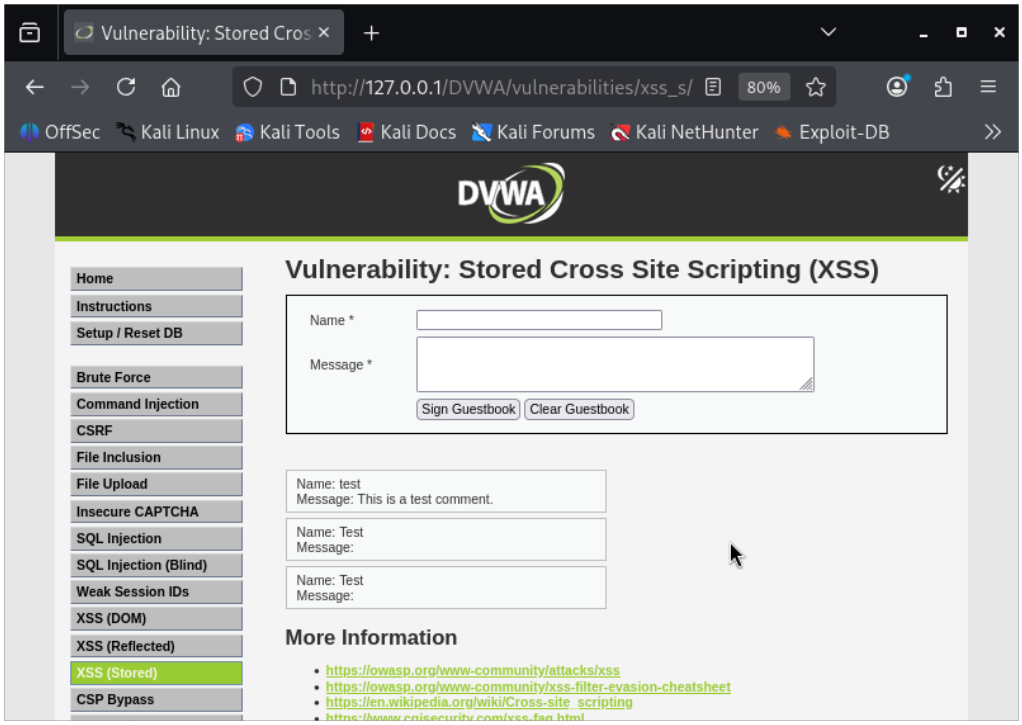


Figure 9. produced during Stored test.

Figure 10. Stored XSS test result in the guestbook

Observed result

The browser displayed an alert after the guestbook entry was submitted. This confirms script execution in the tested Low-security DVWA configuration.

Potential impact

Stored XSS may affect users who later view the affected content. The impact depends on the application, user privileges, and the data or actions accessible in the browser.

Recommended remediation

- Encode untrusted content when displaying it.
- Validate and sanitize input according to its intended format.
- Avoid rendering user input as executable HTML or JavaScript.
- Consider a suitable Content Security Policy.

Task 25 - Directory & Endpoint Discovery

Dirsearch was run against the local DVWA URL to identify accessible directories and files. The scan used the extensions php, html, and txt, with 10 threads.

Command used:

dirsearch -u http://127.0.0.1/DVWA/ -e php,html,txt --threads 10

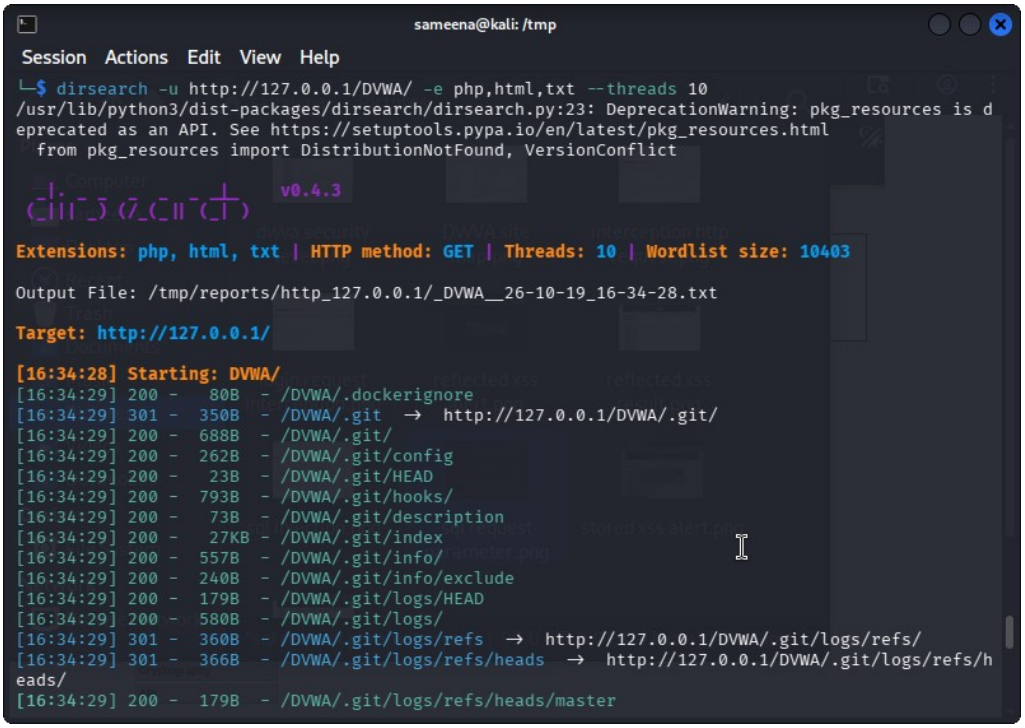


Figure 11. Dirsearch scan results

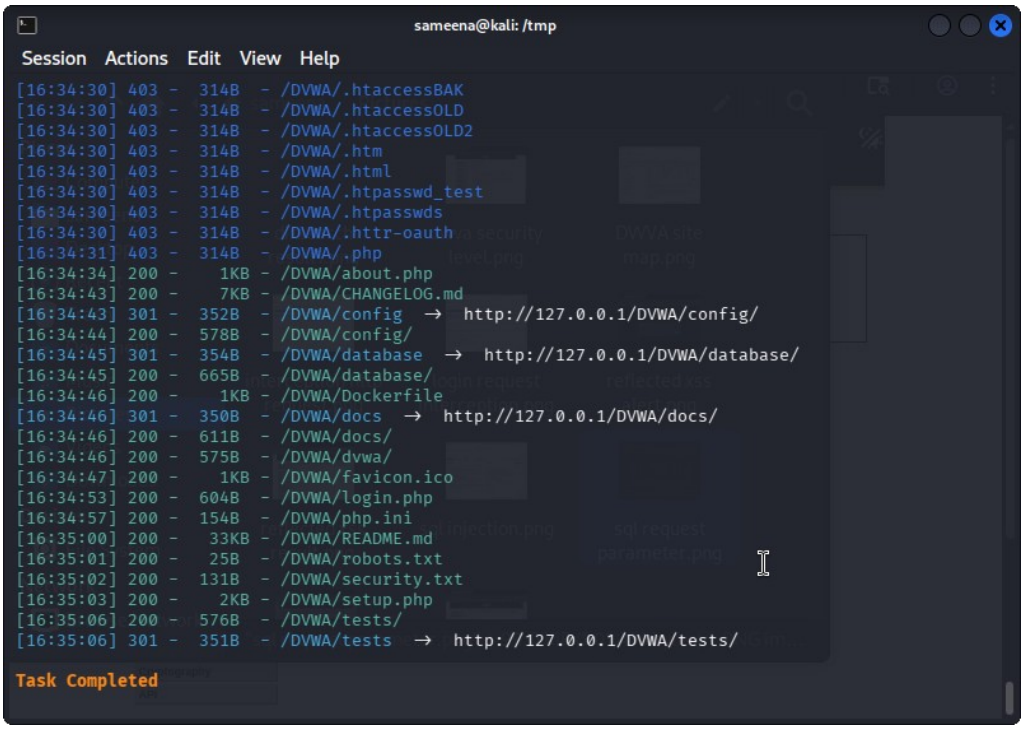


Figure 12. Additional Dirsearch results

Important observations

- Several application paths and files returned HTTP 200 responses.
- The scan reported the .git directory and files such as .git/config as accessible.
- Some tested paths returned HTTP 403 Forbidden.
- The results describe the local lab configuration; each path must be reviewed before classifying it as a securityfinding.

Potential exposure: accessible .git metadata

The scan reported accessible Git metadata beneath the web root. If exposed in a production deployment, Git metadata may reveal repository information or source code. The evidence should be reviewed to confirm what information is actually retrievable.

Recommended remediation

- Do not deploy the .git directory into a public web root.
- Block web access to hidden repository and configuration files.
- Disable unnecessary directory listings.
- Review deployment files and retest after applying changes.

The three confirmed vulnerabilities were reproduced in DVWA, an intentionally vulnerable practice application. The accessible .git paths were observed during enumeration and are recorded separately as a potential information-disclosure issue in the local deployment.

Conclusion

This practical provided hands-on experience with Burp Suite and basic web application security testing. The work included intercepting HTTP requests, mapping the application, inspecting request parameters, testing SQL Injection and XSS in DVWA, and enumerating paths with Dirsearch.

The tests confirmed SQL Injection, Reflected XSS, and Stored XSS in the intentionally vulnerable lab. Directory enumeration also reported accessible .git metadata, which highlights the importance of checking deployment files and restricting access to repository data.

The main learning outcome was understanding how to observe web requests, reproduce selected vulnerabilities, collect evidence, and recommend basic security controls.